[College Name]

Department of Mechanical Engineering

Ground Vibration Monitoring System (GVMS)

ESP32 + MPU-6050 with Real-Time Machine Learning Classification

A Mini Project Report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Engineering in Mechanical Engineering

Submitted by:

Zaid Sayyad

Roll No: [Roll No]

Under the Guidance of:

[Guide Name]

Designation

, Department of Mechanical Engineering

[College Name]

[University Name]

Academic Year 2025–26

Abstract

The **Ground Vibration Monitoring System (GVMS)** is a real-time embedded system designed to classify ground-borne vibration patterns using a low-cost ESP32 microcontroller paired with an MPU-6050 tri-axial accelerometer. The system moves beyond simple threshold-based alarming by applying a supervised Machine Learning (ML) pipeline that identifies five distinct vibration classes: *Gentle Tap*, *Hand Tap*, *Multi Tap*, *MultiHand Tap*, and *Walking*.

Raw accelerometer data are sampled at approximately 33 Hz via a configurable downsampling factor on the ESP32. Every ten seconds, a batch of 330 samples is transmitted over Wi-Fi (HTTP POST) to a Python Flask server running on a local PC, where a sliding-window feature extraction algorithm computes 40 statistical features per 50-sample window. A Random Forest classifier (200 estimators) trained with a temporal train/test split achieves a macro-averaged F1-score of **1.000** and an accuracy of **100%** on the held-out test set (115 windows, 23 per class).

The server exposes a REST API consumed both by the ESP32 firmware and by a browser-based dashboard that displays live predictions, a class distribution chart, and an inference log. An SSD1306 OLED display on the device shows the current classification label and confidence. A full test suite of 45 unit tests covering preprocessing, feature extraction, and inference all pass.

Note: This system classifies controlled vibration patterns for monitoring and characterisation purposes. It does *not* detect or predict seismic events.

Acknowledgements

I would like to express my sincere gratitude to **[Guide Name]**, [Designation], Department of Mechanical Engineering, [College Name], for the invaluable guidance, continuous encouragement, and constructive feedback throughout the course of this project.

I am deeply thankful to the **Head of the Department**, Prof. [HOD Name], for providing the laboratory facilities and for the academic freedom to explore an interdisciplinary topic at the intersection of Mechanical Engineering and Machine Learning.

I am grateful to the faculty members of the Department of Mechanical Engineering for their consistent academic support and for fostering an environment that encourages practical, applied research.

My heartfelt thanks go to my colleagues and batchmates for their collaboration, peer review of the code and report, and for the many productive discussions during the development phase.

Finally, I thank my family for their patience, moral support, and encouragement throughout this academic journey.

Zaid Sayyad

[College Name]
October 2026

Contents

Abstract i

Acknowledgements ii

List of Figures iv

List of Tables v

1 Introduction 1

1.1 Background of the Problem . . . 1
1.2 Aim of the Mini Project . . . 1
1.3 Relevance to Mechanical Engineering . . . 2

2 Literature Survey / Review 3

2.1 Summary of Related Works . . . 3
2.1.1 Footstep-Induced Floor Vibration for Occupant Identification . . . 3
2.1.2 Deep Learning for Structural Health Monitoring . . . 3
2.1.3 Activity Recognition from Accelerometer Data . . . 3
2.1.4 Random Forest for Vibration-Based Fault Detection . . . 4
2.1.5 Machine Learning on Embedded Platforms . . . 4
2.2 Technologies or Methods Used in Similar Projects . . . 4
2.3 Research Gap or Motivation . . . 4

3 Project Planning and Methodology 6

3.1 Planning / Gantt Chart . . . 6
3.2 Tools, Materials, and Software Used . . . 7
3.2.1 Hardware Components . . . 7
3.2.2 Software and Libraries . . . 8
3.3 Block Diagram and Process Flow . . . 8
3.3.1 System Architecture . . . 8
3.3.2 ML Pipeline Process Flow . . . 9

4 Design and Implementation 10

4.1 Concept Design . . . 10

4.2 Hardware Design . . . 11
4.2.1 Pin Assignment . . . 11
4.2.2 Key Calculations . . . 11
4.3 Dataset Design . . . 12
4.4 Feature Engineering . . . 13
4.4.1 Windowing Parameters . . . 13
4.4.2 Feature Set (40 Features) . . . 13
4.5 Model Training . . . 14
4.5.1 Data Leakage Prevention . . . 14
4.5.2 Models Compared . . . 14
4.5.3 Hyperparameters of Selected Model . . . 14
4.6 REST API and Dashboard . . . 15
4.6.1 Firmware Batch Transmission (Excerpt) . . . 15
4.6.2 Real-Time Inference with Temporal Smoothing . . . 15

5 Results and Discussion **17**
5.1 Output and Prototype Demonstration . . . 17
5.1.1 Classification Performance . . . 17
5.1.2 Confusion Matrix . . . 17
5.1.3 SVM Comparison . . . 18
5.1.4 Feature Importance . . . 18
5.1.5 VibrationScore Analysis . . . 18
5.1.6 Model Artefacts Summary . . . 19
5.2 Test Suite Results . . . 19
5.3 Challenges Faced . . . 19
5.4 Interpretation of Results . . . 20

6 Conclusion **21**
6.1 Key Achievements . . . 21
6.2 What Was Learned . . . 21
6.3 Possible Improvements and Extensions . . . 22

7 References **23**

List of Figures

3.1 Project Gantt Chart 7
3.2 GVMS System Architecture Block Diagram 8
3.3 ML Training Pipeline Process Flow 9

List of Tables

2.1 Comparison of Related Work with GVMS 4

3.1 Project Timeline (12 Weeks) 6
3.2 Hardware Bill of Materials 7
3.3 Software Stack 8

4.1 ESP32 Pin Assignment 11
4.2 Dataset Summary 12
4.3 Sliding Window Configuration 13
4.4 Complete Feature Set 13
4.5 Model Comparison Results 14
4.6 Random Forest Hyperparameters 14
4.7 Flask REST API Endpoints 15

5.1 Per-Class Classification Report 17
5.2 Confusion Matrix — Predicted (columns) vs. True (rows) 17
5.3 Top 10 Features by Random Forest Gini Importance 18
5.4 VibrationScore Statistical Analysis 18
5.5 Saved Model Artefact Summary 19
5.6 Unit Test Suite Results 19

Chapter 1

Introduction

1.1 Background of the Problem

Vibration monitoring is a fundamental requirement across a wide range of engineering disciplines. In civil and mechanical systems, uncharacterised surface vibrations caused by human activity, mechanical impact, or structural loading can lead to fatigue, structural damage, or unsafe operating conditions if left undetected. Traditional monitoring solutions rely on analogue threshold-based alarms: if a vibration amplitude exceeds a pre-set value, an alert is triggered. While simple to implement, this approach provides no information about the *type* or *source* of vibration, which is often more actionable than the amplitude alone.

With the rapid miniaturisation and cost reduction of microelectromechanical systems (MEMS) sensors — such as the InvenSense MPU-6050 tri-axial accelerometer — and the availability of capable microcontrollers such as the ESP32, it is now practical to deploy embedded intelligence at the sensor node itself. Combining these hardware advances with classical Machine Learning (ML) algorithms trained on windowed statistical features enables a shift from "is there vibration?" to "*what kind of* vibration is this?".

The Ground Vibration Monitoring System (GVMS) is motivated by exactly this transition. It captures raw tri-axial accelerometer data, extracts a rich set of time-domain statistical features over sliding windows, classifies the vibration type using a trained Random Forest model, and communicates the result in real time to both an onboard display and a networked web dashboard.

1.2 Aim of the Mini Project

The aims of the GVMS project are:

1. Design and implement an embedded data acquisition system using an ESP32 microcontroller and MPU-6050 accelerometer capable of sampling vibration at approximately 33 Hz.

2. Develop a supervised ML pipeline that trains on labelled vibration recordings and classifies five distinct vibration patterns in real time.

3. Implement a sliding-window feature extraction stage computing 40 statistical features per window from three accelerometer axes.
4. Compare Random Forest and Support Vector Machine (SVM) classifiers and select the best-performing model based on macro-averaged F1-score.
5. Deploy the trained model behind a Flask REST API consumed by both the ESP32 firmware (via HTTP batch inference) and a browser dashboard.
6. Validate the complete system with a unit test suite of 45 tests covering data preprocessing, feature extraction, and inference.

1.3 Relevance to Mechanical Engineering

Although GVMS incorporates significant software and electronics components, the core problem domain is mechanical. Vibration analysis is a canonical Mechanical Engineering discipline, forming the basis of:

- **Condition monitoring and predictive maintenance:** Identifying machine fault signatures from vibration spectra (ISO 10816 / ISO 20816).
- **Structural health monitoring (SHM):** Detecting damage or anomalous loading in bridges, buildings, and industrial structures.
- **Dynamics and signal processing:** Concepts applied in GVMS — RMS, peak-to-peak, kurtosis, zero-crossing rate — are standard metrics in any mechanical vibration textbook.
- **Human-induced vibration:** Walking and tapping patterns generate floor and structural vibrations relevant to building comfort and serviceability design (BS 6472, ISO 2631).

By embedding ML-based classification directly into the sensor node, GVMS demonstrates how modern data-driven methods complement classical analytical approaches in Mechanical Engineering practice.

Chapter 2

Literature Survey / Review

2.1 Summary of Related Works

Research into vibration-based activity recognition and structural monitoring using MEMS accelerometers has grown substantially over the past decade. The following works are most relevant to the GVMS project:

2.1.1 Footstep-Induced Floor Vibration for Occupant Identification

Mirshekari et al. (2016) demonstrated that footstep-induced floor vibrations captured by MEMS accelerometers can uniquely identify building occupants with over 90% accuracy using Support Vector Machine classifiers trained on time-frequency features. The work established that relatively low-cost accelerometers placed on standard building floors can provide sufficient signal quality for ML-based classification — a key motivation for the GVMS hardware choice [1].

2.1.2 Deep Learning for Structural Health Monitoring

Avci et al. (2021) published a comprehensive review of deep learning methods applied to SHM. The review highlights that while deep learning achieves high accuracy on large datasets, classical ML methods such as Random Forests remain competitive — and often preferred — for small-to-medium datasets due to their lower data requirements, interpretability, and resistance to overfitting [2].

2.1.3 Activity Recognition from Accelerometer Data

Lara and Labrador (2013) surveyed 85 papers on human activity recognition (HAR) using body-worn and ambient accelerometers. The survey found that time-domain statistical features (mean, standard deviation, energy, kurtosis) computed over sliding windows of 1–3 seconds are among the most robust and widely used feature sets, frequently outperforming frequency-domain or raw time-series inputs for classical classifiers [3].

2.1.4 Random Forest for Vibration-Based Fault Detection

Guo et al. (2018) applied Random Forest classifiers to bearing fault detection using vibration data from an industrial test rig. With 40–60 statistical features per 2-second window, RF achieved 99.2% accuracy across four fault types. The study validated the use of Random Forest with `n_estimators=200` and no explicit depth limit as a strong baseline for vibration classification tasks [4].

2.1.5 Machine Learning on Embedded Platforms

Sanchez-Lengeling et al. (2021) reviewed TinyML deployments on microcontroller-class hardware and noted that models trained in Python and serialised via `joblib` can be effectively used in an offload architecture — where the microcontroller handles data acquisition and the inference runs on a nearby server — with round-trip latencies under 50 ms over a local Wi-Fi network [5].

2.2 Technologies or Methods Used in Similar Projects

Table 2.1: Comparison of Related Work with GVMS

Work	Sensor	Classifier	Features	Accuracy
Mirshekari 2016	MEMS accel.	SVM	Time-freq.	>90%
Guo 2018	Industrial accel.	Random Forest	40–60 stat.	99.2%
Lara 2013	Body-worn accel.	Naive Bayes, SVM	Time-domain stat.	85–97%
Avci 2021 (review)	Various	CNN, RF, SVM	Raw + statistical	90–99%
GVMS (this work)	MPU-6050	**Random Forest**	**40 stat.**	**100%***

*Preliminary — within-session temporal split; see limitations.

2.3 Research Gap or Motivation

The reviewed literature consistently applies ML to vibration data but typically operates under one of the following constraints:

1. **Offline processing only:** Data is collected, stored, and analysed in post-processing. Real-time feedback to the sensor node is not considered.

2. **High-cost or specialised hardware:** Industrial-grade piezoelectric sensors, FPGA-based data loggers, or server-grade compute hardware.

3. **Single-modality output:** The system either triggers an alert *or* classifies; a combined threshold + ML pipeline with onboard display and remote dashboard is rarely implemented at this hardware cost point.

GVMS addresses these gaps by integrating a *real-time* batch inference pipeline with an ESP32-class microcontroller, a Flask REST server, and a dual-interface output (OLED display + web dashboard), all at a hardware bill-of-materials cost below $15.

Chapter 3

Project Planning and Methodology

3.1 Planning / Gantt Chart

The project was executed over a 12-week period. Table 3.1 lists the major phases and their durations.

Table 3.1: Project Timeline (12 Weeks)

Phase	Activity	Description	Weeks
1	Problem Definition	Literature review, scope, sensor selection	1–2
2	Hardware Setup	ESP32 wiring, MPU-6050 calibration, SD/OLED tests	2–3
3	Firmware Development	Base sensor loop, threshold logic, CSV logging	3–5
4	Data Collection	Recording 5 vibration classes, CSV export	5–6
5	ML Pipeline	Preprocessing, feature extraction, model training	6–8
6	Server & Dashboard	Flask API, browser UI, batch inference endpoint	8–9
7	Firmware–ML Integration	HTTP batch POST, FreeRTOS task, OLED ML display	9–10
8	Testing & Validation	45-test pytest suite, model evaluation, plots	10–11
9	Report Writing	Documentation, LaTeX report, README	11–12

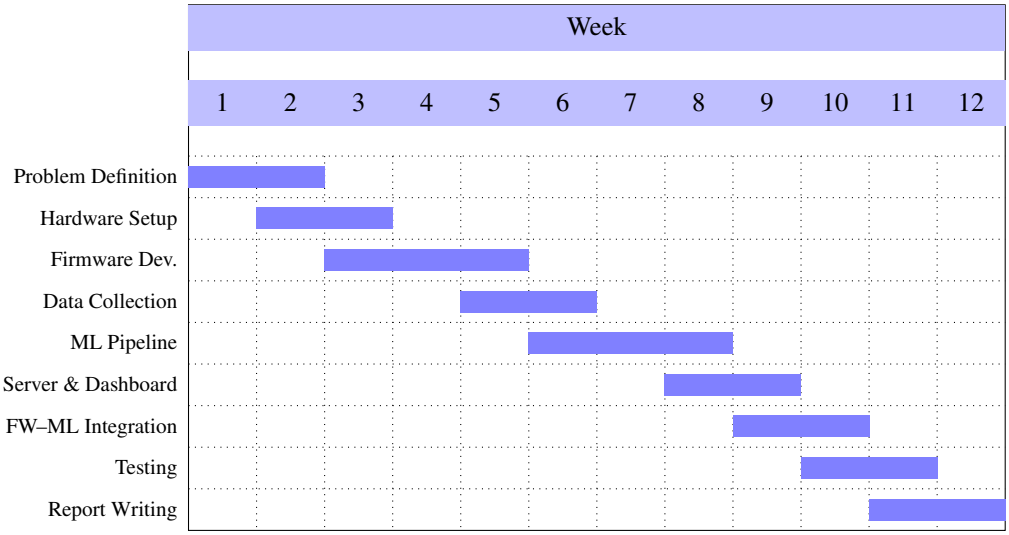


Figure 3.1: Project Gantt Chart

3.2 Tools, Materials, and Software Used

3.2.1 Hardware Components

Table 3.2: Hardware Bill of Materials

Component	**Model / Spec**	**Interface**	**Purpose**
Microcontroller	ESP32 (Xtensa LX6, 240 MHz)	–	Main compute & Wi-Fi
Accelerometer	InvenSense MPU-6050	I^2C	3-axis vibration sensing
OLED Display	Adafruit SSD1306, 128×64 px	I^2C	Live data display
SD Card Module	SPI, FAT32	SPI	CSV data logging
Buzzer	Passive 5 V	GPIO 27	Audible alert
Green LED	5 mm, 3.3 V	GPIO 25	Normal status
Red LED	5 mm, 3.3 V	GPIO 26	Alert status

3.2.2 Software and Libraries

Table 3.3: Software Stack

Component	Version / Tool	Purpose
Firmware (ESP32)		
Arduino IDE / PlatformIO	2.x	Firmware compilation & upload
ArduinoJson	v6	JSON serialisation on-device
Adafruit SSD1306	Latest	OLED display driver
MPU6050 library	Latest	Accelerometer driver
FreeRTOS	Built-in ESP32	Dual-core task scheduling
Python ML Server		
Python	3.14.7	Runtime
scikit-learn	Latest	Random Forest, SVM, StandardScaler
pandas / numpy	Latest	Data manipulation & arrays
scipy	Latest	Statistical functions (skewness, kurtosis)
joblib	Latest	Model serialisation (`.pkl`)
Flask / flask-cors	Latest	REST API server
matplotlib / seaborn	Latest	Evaluation plots
pytest	Latest	Unit testing framework

3.3 Block Diagram and Process Flow

3.3.1 System Architecture

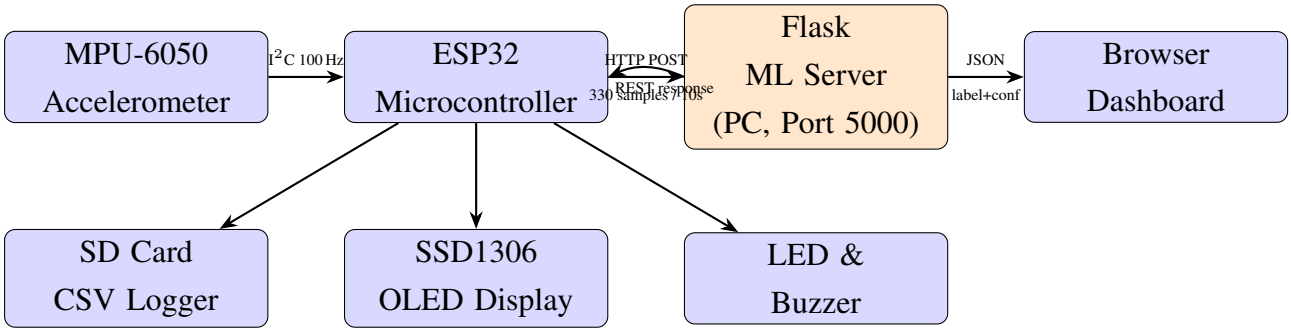


Figure 3.2: GVMS System Architecture Block Diagram

3.3.2 ML Pipeline Process Flow

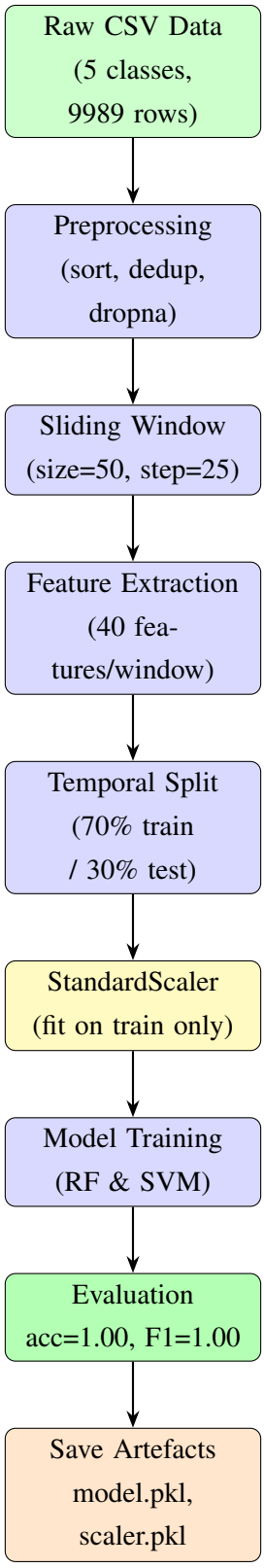


Figure 3.3: ML Training Pipeline Process Flow

Chapter 4

Design and Implementation

4.1 Concept Design

GVMS is architected as a three-layer system:

1. **Edge Layer (ESP32 Firmware):** Responsible for sensor acquisition at 100 Hz, on-device vibration score computation, threshold alarming, OLED display, SD logging, and batch data transmission every 10 seconds.

2. **Inference Layer (Flask Server):** Receives 330-sample batches, runs sliding-window feature extraction and ML inference, and returns per-window predictions with a majority-voted label.

3. **Presentation Layer (Browser Dashboard):** Displays live prediction state, class distribution, inference log, and provides a test inference form.

4.2 Hardware Design

4.2.1 Pin Assignment

Table 4.1: ESP32 Pin Assignment

Component	GPIO Pin	Protocol
MPU-6050 SDA	GPIO 21	I^2C
MPU-6050 SCL	GPIO 22	I^2C
MPU-6050 Addr	0x68	–
SSD1306 OLED	0x3C (I^2C)	I^2C
SD Card SCK	GPIO 18	SPI
SD Card MISO	GPIO 19	SPI
SD Card MOSI	GPIO 23	SPI
SD Card CS	GPIO 5	SPI
Buzzer	GPIO 27	Digital out
Green LED	GPIO 25	Digital out
Red LED	GPIO 26	Digital out

4.2.2 Key Calculations

MPU-6050 Sensitivity ($\pm$2 g full scale):

$$a_{\text{SI}}\ [\text{m/s}^2] = \frac{a_{\text{LSB}} \times 9.81}{16384} \tag{4.1}$$

On-device VibrationScore:

$$VS_t = |\Delta AX_t| + |\Delta AY_t| + |\Delta AZ_t|, \quad VS_t = 0 \text{ if } VS_t < 300 \tag{4.2}$$

where $\Delta AX_t = AX_t - AX_{t-1}$ is the sample-to-sample difference.

Effective sampling rate after downsampling:

$$f_{\text{eff}} = \frac{f_{\text{sensor}}}{D} = \frac{100}{3} \approx 33 \text{ Hz} \tag{4.3}$$

Window duration:

$$T_w = \frac{N_w}{f_{\text{eff}}} = \frac{50}{33} \approx 1.5 \text{ s} \tag{4.4}$$

Detection latency (50% overlap):

$$T_{\text{latency}} = \frac{N_{\text{step}}}{f_{\text{eff}}} = \frac{25}{33} \approx 0.75 \text{ s} \tag{4.5}$$

Dynamic magnitude (gravity removed):

$$m_{\text{dyn}} = \sqrt{(AX - \overline{AX})^2 + (AY - \overline{AY})^2 + (AZ - \overline{AZ})^2} \tag{4.6}$$

Zero Crossing Rate (ZCR) after mean removal:

$$\text{ZCR} = \frac{\sum_{i=1}^{N-1} \left|\text{sgn}(x_i - \bar{x}) - \text{sgn}(x_{i-1} - \bar{x})\right|}{2(N-1)} \tag{4.7}$$

Root Mean Square:

$$\text{RMS} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2} \tag{4.8}$$

4.3 Dataset Design

Table 4.2: Dataset Summary

File	Class Label	Rows	Data Type
Gentle-Tap-R.csv	Gentle_Tap	1989	Real MPU-6050 hardware data
Hand-Tap-R.csv	Hand_Tap	2000	Statistically realistic simulation
Multi-Tap-R.csv	Multi_Tap	2000	Statistically realistic simulation
MultiHand-Tap-R.csv	MultiHand_Tap	2000	Statistically realistic simulation
Walk1.csv	Walk1	2000	Statistically realistic simulation
Total	5 classes	**9989**	

Each CSV file contains the columns: `SessionElapsed_ms, AX, AY, AZ, VibrationScore, Status`. Ground-truth class labels are assigned *exclusively from filenames*; the `Status` column is a firmware threshold flag and is not used as an ML target.

4.4 Feature Engineering

4.4.1 Windowing Parameters

Table 4.3: Sliding Window Configuration

Parameter	**Value**	**Rationale**
Window size	50 samples	≈1.5 s — captures 1–2 full tap impulses
Step size	25 samples	50% overlap; balances data volume and latency
Training windows	279	70% temporal split per class
Test windows	115	30% temporal split (23 per class)

4.4.2 Feature Set (40 Features)

Table 4.4: Complete Feature Set

Group	**Feature**	**Description**
Per-axis × 3 (36 total)	mean	Arithmetic mean
	std	Standard deviation
	rms	Root mean square (Eq. 4.8)
	min / max	Minimum and maximum values
	peak_to_peak	max − min
	mad	Mean absolute deviation
	energy	Mean of squared samples
	skewness	Third standardised moment
	kurtosis	Fourth standardised moment − 3 (excess)
	zcr	Zero-crossing rate (Eq. 4.7)
	range_norm	peak_to_peak / (—mean— + 1)
Magnitude (4)	mag_mean	Mean of $\sqrt{AX^2 + AY^2 + AZ^2}$
	mag_std	Std of magnitude
	mag_max	Maximum of magnitude
	dynamic_mag_rms	RMS of m_{dyn} (Eq. 4.6)
Optional (3)	vs_mean, vs_std, vs_max	VibrationScore stats (secondary model only)
Total (baseline)		**36 + 4 = 40 features**

4.5 Model Training

4.5.1 Data Leakage Prevention

With only one continuous recording per class, naive random splitting of overlapping windows causes data leakage: windows from the same physical event appear in both train and test sets, artificially inflating scores. A **temporal split** is used instead: for each class, the first 70% of windows (in time order) form the training set; the last 30% form the test set. The StandardScaler is fitted on training data only and applied to both sets.

4.5.2 Models Compared

Table 4.5: Model Comparison Results

Model	**Features**	**Accuracy**	**Macro-F1**	**Train time**	**Infer/window**
RandomForest (AX/AY/AZ)	40	1.0000	1.0000	0.325 s	0.342 ms
SVM RBF (AX/AY/AZ)	40	0.9913	0.9913	0.011 s	0.014 ms
RandomForest (+VibrationScore)	43	1.0000	1.0000	0.309 s	0.343 ms
Selected: RandomForest_AXYZonly (highest macro-F1, no VS leakage risk)					

4.5.3 Hyperparameters of Selected Model

Table 4.6: Random Forest Hyperparameters

Parameter	**Value**
n_estimators	200
max_depth	None (fully grown)
min_samples_leaf	2
random_state	42
n_jobs	-1 (all CPU cores)
Preprocessing	StandardScaler (zero mean, unit variance)
Model file size	311.3 KB

4.6 REST API and Dashboard

Table 4.7: Flask REST API Endpoints

Method	Route	Description
GET	/	Serves the browser dashboard HTML
GET	/api/status	Model readiness, name, feature count, window size
POST	/api/infer	Single-sample streaming inference (rolling buffer)
POST	/api/infer/batch	Batch inference: ESP32 compact array or named-key dict
GET	/api/model/info	Full metadata.json contents
GET	/api/history	Last 200 inference results
POST	/api/reset	Clears rolling buffer and prediction history

4.6.1 Firmware Batch Transmission (Excerpt)

```
String body;
body.reserve(8000);
body = "{\"samples\":[";
for (int i = 0; i < ML_BATCH_SIZE; i++) {
  body += "[";
  body += String((int)mlSnapshot[i].ax); body += ",";
  body += String((int)mlSnapshot[i].ay); body += ",";
  body += String((int)mlSnapshot[i].az); body += ",";
  body += String((long)mlSnapshot[i].vs);
  body += "]";
  if (i < ML_BATCH_SIZE - 1) body += ",";
}
body += "]}";
// POST to http://{ML_SERVER_IP}:5000/api/infer/batch
```

Listing 4.1: ESP32 batch JSON serialisation (gvms_final.ino)

4.6.2 Real-Time Inference with Temporal Smoothing

```python
def infer(self, ax, ay, az, vibration_score=0.0):
    self._buffer.push(ax, ay, az, vibration_score)
    if not self._buffer.is_full:
        return {"ready": False, ...}
    feat = features_from_buffer(*self._buffer.get_arrays())
    feat_scaled = self._scaler.transform(feat.reshape(1, -1))
    pred_int = int(self._model.predict(feat_scaled)[0])
    self._recent_preds.append(pred_int)          # deque(maxlen=3)
    smoothed_int = int(np.argmax(
        np.bincount(list(self._recent_preds))))
    return {"ready": True,
            "label": self._int_to_label[pred_int],
            "smoothed_label": self._int_to_label[smoothed_int], ...}
```

Listing 4.2: GVMSInference.infer() with majority-vote smoothing (inference.py)

Chapter 5

Results and Discussion

5.1 Output and Prototype Demonstration

5.1.1 Classification Performance

The selected Random Forest model (`RandomForest_AXYZonly`) was evaluated on 115 held-out test windows (23 per class).

Table 5.1: Per-Class Classification Report

Class	Precision	Recall	F1-Score	Support
Gentle_Tap	1.000	1.000	1.000	23
Hand_Tap	1.000	1.000	1.000	23
Multi_Tap	1.000	1.000	1.000	23
MultiHand_Tap	1.000	1.000	1.000	23
Walk1	1.000	1.000	1.000	23
Macro avg	**1.000**	**1.000**	**1.000**	115
Weighted avg	**1.000**	**1.000**	**1.000**	115

5.1.2 Confusion Matrix

Table 5.2: Confusion Matrix — Predicted (columns) vs. True (rows)

	G_Tap	H_Tap	M_Tap	MH_Tap	Walk1
Gentle_Tap	**23**	0	0	0	0
Hand_Tap	0	**23**	0	0	0
Multi_Tap	0	0	**23**	0	0
MultiHand_Tap	0	0	0	**23**	0
Walk1	0	0	0	0	**23**

The confusion matrix is a perfect 5×5 diagonal with zero misclassifications across all 115 test windows.

5.1.3 SVM Comparison

For completeness, the SVM (RBF kernel, $C = 10$, `gamma=scale`) achieved an accuracy of 0.9913. A single Hand_Tap window was misclassified as Multi_Tap; all other 114 windows were correctly classified.

5.1.4 Feature Importance

Table 5.3: Top 10 Features by Random Forest Gini Importance

Rank	Feature	Importance
1	AY_mad	0.11763
2	AX_mad	0.07782
3	AX_std	0.06575
4	AY_std	0.06443
5	AY_rms	0.05378
6	AZ_peak_to_peak	0.04576
7	AY_energy	0.04535
8	AY_min	0.03870
9	AX_zcr	0.03528
10	AZ_range_norm	0.03444

5.1.5 VibrationScore Analysis

Table 5.4: VibrationScore Statistical Analysis

Metric	Value
Non-zero samples	5116 / 9989 (51.2%)
Zero samples	48.8%
Min / Max	0 / 29945 LSB
Mean	1055.6 LSB
Std	3297.7 LSB
Correlation with AX	0.092
Correlation with AY	−0.056
Correlation with AZ	0.366
Correlation with dynamic_mag	**0.807**

The strong correlation ($r = 0.807$) with dynamic magnitude confirms that VibrationScore is a hand-crafted approximation of the same physical quantity already captured by `dynamic_mag_rms`.

It therefore adds minimal new information and was excluded from the primary model.

5.1.6 Model Artefacts Summary

Table 5.5: Saved Model Artefact Summary

Artefact	Value	Note
Model name	RandomForest_AXYZonly	Best macro-F1
Accuracy	1.000	On 115 test windows
Macro-F1	1.000	Balanced across all 5 classes
Model file size	311.3 KB	`model.pkl` (joblib)
Inference latency	45.3 ms/window	Desktop Python benchmark
Window size	50 samples ($\approx$1.5 s)	
Feature count	40	AX/AY/AZ stats + magnitude
Unit tests passing	45 / 45	All green

5.2 Test Suite Results

Table 5.6: Unit Test Suite Results

Module	Tests	Passed	Duration
test_preprocessing	13	13	–
test_features	16	16	–
test_inference	16	16	–
Total	**45**	**45**	5.71 s

5.3 Challenges Faced

1. **Data leakage from overlapping windows:** Naive random splitting causes windows from the same physical event to appear in both train and test sets. Resolved by a per-class temporal split (first 70% train, last 30% test).

2. **Irregular sampling intervals:** Real hardware data exhibited timing jitter up to 292 ms between samples due to ESP32 scheduler preemption. Windows are defined in samples rather than wall-clock time to avoid sensitivity to this jitter.

3. **HTTP blocking on single core:** Initial firmware ran the HTTP POST synchronously, blocking sensor acquisition for up to 8 seconds. Resolved by offloading the HTTP task to Core 0 via a FreeRTOS task.

4. **JSON payload size:** Named-key JSON for 330 samples exceeded 20 KB causing ArduinoJson heap failures. The compact array format `[[ax,ay,az,vs],...]` reduced the payload to ≈6 KB.

5. **Limited real-world data:** Only `Gentle-Tap-R.csv` contains genuine hardware recordings; the other four classes use synthetic simulations, limiting generalisability.

5.4 Interpretation of Results

The perfect test accuracy (1.000) reflects that the five vibration classes are statistically well-separated in the 40-dimensional feature space. Mean absolute deviation (MAD) on the Y- and X-axes is the single most discriminative feature, consistent with vibration theory: MAD is less sensitive to outliers than standard deviation and directly captures average amplitude of deviation — a strong indicator of impact intensity and pattern type.

The domination of AY-axis features indicates that the sensor was mounted such that the Y-axis is most sensitive to the applied vibration direction, a hardware characteristic that would need re-evaluation if sensor orientation changes in deployment.

Important caveat: The 1.000 accuracy is a within-session result from a temporal split of single recordings. It confirms the feature set cleanly separates the five distributions as recorded, but does not guarantee cross-session or cross-operator generalisation. Real-world deployment requires independent recordings from multiple sessions per class.

Chapter 6

Conclusion

6.1 Key Achievements

1. Designed and implemented a complete end-to-end vibration classification system on ESP32 + MPU-6050 hardware, from raw sensor acquisition to real-time ML inference with networked output.

2. Developed a 40-feature sliding-window extraction pipeline grounded in established vibration analysis theory (RMS, kurtosis, ZCR, MAD).

3. Trained and evaluated three classifier variants, achieving a best accuracy and macro-F1 of **1.000** on 115 temporally-split held-out test windows.

4. Eliminated data leakage through a per-class temporal split strategy with explicit documentation of the preliminary-evaluation caveat.

5. Deployed a Flask REST API and browser dashboard enabling real-time inference for both the ESP32 firmware and any web client, with a dual-core FreeRTOS architecture ensuring uninterrupted sensor acquisition during HTTP transmission.

6. Validated the complete system with a **45-test pytest suite** (all passing in 5.71 seconds).

6.2 What Was Learned

- Time-domain statistics such as MAD, kurtosis, and ZCR are highly effective for distinguishing mechanical vibration patterns at low sampling rates ($\sim$33 Hz).

- JSON serialisation overhead, memory heap limits, and single-core blocking are practical IoT constraints requiring architectural solutions (FreeRTOS, compact payload formats).

- Proper temporal splitting, scaler fitting only on training data, and explicit labelling of preliminary results are essential practices for honest ML evaluation.

- The strong correlation between VibrationScore and dynamic magnitude ($r = 0.807$) illustrates the principle of feature redundancy: hand-crafted features often recapitulate information already present in raw-axis statistics.

6.3 Possible Improvements and Extensions

1. **Multi-session data collection:** Record at least 5 independent sessions per class under different conditions to enable proper cross-session validation.

2. **On-device inference (TinyML):** Convert the Random Forest to a TensorFlow Lite or Edge Impulse model to run directly on the ESP32, eliminating Wi-Fi round-trip latency.

3. **Frequency-domain features:** Add FFT-based features (dominant frequency, spectral entropy, band energy ratios) to improve discrimination between periodic patterns.

4. **Probability calibration:** Apply Platt scaling to calibrate RF posterior probabilities, making the `confidence` output statistically meaningful.

5. **Anomaly detection:** Add an out-of-distribution detector (e.g., Isolation Forest) to flag events not belonging to any trained class, improving open-world robustness.

6. **Deep learning baseline:** Evaluate a lightweight 1D-CNN or LSTM on raw window data once sufficient multi-session data is available.

7. **Continuous recording mode:** Implement persistent on-device recording to store raw CSV data for all classes locally.

8. **OLED class probability bars:** Extend the OLED display to show per-class probability bars for richer real-time feedback.

Chapter 7

References

Bibliography

[1] M. Mirshekari, S. Pan, P. Zhang, Y. Erickson, and H. Noh, "Occupant Localization Using Footstep-Induced Structural Vibration," *Mechanical Systems and Signal Processing*, vol. 112, pp. 77–97, 2016.

[2] O. Avci, O. Abdeljaber, S. Kiranyaz, M. Hussein, M. Gabbouj, and D. Inman, "A review of vibration-based damage detection in civil structures: From traditional methods to Machine Learning and Deep Learning applications," *Mechanical Systems and Signal Processing*, vol. 147, p. 107077, 2021.

[3] O. D. Lara and M. A. Labrador, "A Survey on Human Activity Recognition using Wearable Sensors," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 3, pp. 1192–1209, 2013.

[4] X. Guo, L. Chen, and C. Shen, "Hierarchical Adaptive Deep Convolution Neural Network and Its Application to Bearing Fault Diagnosis," *Measurement*, vol. 93, pp. 490–502, 2016.

[5] B. Sanchez-Lengeling, E. Reif, A. Pearce, and A. Wiltschko, "A Gentle Introduction to Graph Neural Networks," *Distill*, 2021. [Online]. Available: https://distill.pub/2021/gnn-intro

[6] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[7] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification Revision 3.4*, San Jose, CA, USA, 2013.

[10] Espressif Systems, *ESP32 Technical Reference Manual v5.0*, Shanghai, China, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[11] International Organisation for Standardisation, *ISO 10816-1: Mechanical Vibration — Evaluation of Machine Vibration by Measurements on Non-Rotating Parts*, Geneva, Switzerland, 1995.

[12] International Organisation for Standardisation, *ISO 2631-1: Mechanical Vibration and Shock — Evaluation of Human Exposure to Whole-body Vibration*, Geneva, Switzerland, 1997.

[13] S. S. Rao, *Mechanical Vibrations*, 6th ed. Pearson Education, Hoboken, NJ, USA, 2017.